

基于 UnifiedReward-qwen 文生图强化学习实验报告

1 项目背景

近年来，扩散模型（Diffusion Models）在文本生成图像（Text-to-Image, T2I）任务中表现出色。特别是 Stable Diffusion 3.5 (SD3.5) 在生成质量和鲁棒性上相比前代模型有明显提升。然而，单纯依赖提示词（prompt）的生成过程存在探索效率不足、收敛次优等问题。为进一步提升模型的表现，业界提出了基于 强化学习（RL）+ 偏好优化（DPO, Direct Preference Optimization） 的训练框架，通过奖励模型（如 UnifiedReward-Qwen）对生成结果进行评估，引导扩散模型在图像质量、语义对齐等指标上进一步优化。

本实验旨在 2*3090（24GB）下，对 SD3.5 medium + LoRA 适配器 进行训练与推理加速，结合 Geneval 指标评测 与 UnifiedReward-Qwen 奖励模型，实现一个完整的文生图强化学习优化与评估流程。

2 研究目标

- (1) 搭建 Stable Diffusion 3.5 Large/Medium 的推理与训练流水线；
- (2) 基于 LoRA（Low-Rank Adaptation） 微调方式，降低显存消耗；
- (3) 在 多 GPU 环境 下实现 任务并行/数据并行，缩短推理时间；
- (4) 结合 UnifiedReward-Qwen 奖励模型 完成偏好对比数据构建与 DPO 优化；
- (5) 使用 Geneval 框架完成客观指标评测，对比基线模型与优化模型。

3 实验过程

3.1 偏好数据构造

该部分实现主要依靠 <https://github.com/Tencent-Hunyuan/UnifiedReward> 中的使用方法实现。

- (1) 生成偏好数据

用 UnifiedReward 模型对不同 prompt 下生成的图片对（A vs B）进行打分
→ 得到“哪张更好”的偏好标签。

(2) pair → point 转换

把成对的偏好数据转成单样本数据（比如好图 → 正例，差图 → 负例），得到 DPO 可用的数据格式。

得到如下格式的 json 文件：

```
[
  {
    "id": "",
    "caption": "",
    "jpg_0": "", #chosen image path
    "jpg_1": "", #rejected image path
    "label_0": 1,
  },
  ...
]
```

3.2 常见工具与算法

3.2.1 常见算法

(1) PPO（Proximal Policy Optimization）

优点：

- a. 经典 RLHF 方法，理论和实现都很成熟。
- b. 能够利用 reward model 的连续信号（不仅仅是偏好对），可做更精细的优化。
- c. 在语言模型 RLHF（ChatGPT）中应用最广。

缺点：

- a. 训练复杂：需要引入 reward model 参与每一步训练，reward model 和生成模型要同时 forward，计算量大。
- b. 不稳定：容易 collapse，需要 KL 正则约束。

c. 显存和算力消耗大：对 diffusion 这种本就重的模型来说几乎不可承受。

(2) GRPO (Group Relative Policy Optimization)

优点：

a. 对比 PPO，收敛更快：直接比较一组候选，利用“胜者相对劣者”的信号优化。

b. 适合多样化场景，不必严格依赖 reward model。

c. 更适合视觉场景（因为一组图里好坏对比往往比绝对 reward 分数更直观）。

缺点：

a. 训练仍然复杂：要采样多张候选图（batch 内部要分组）。

b. 算力消耗大：比 PPO 略轻，但对 diffusion 仍然昂贵。

c. 实现难度高，尤其是需要 carefully 设计分组和采样策略。

(3) DPO (Direct Preference Optimization)

优点：

a. 不需要 reward model 参与训练：reward model 只在数据预处理时打分，训练时直接用 pair 数据。

b. 训练稳定：loss 类似 logistic，单调可控，不需要 KL 正则。

c. 效率高：比 PPO/GRPO 都快很多，更适合大模型+低秩微调（LoRA）。

d. 对分布式友好：结合 VERL 可以 scale 到多卡。

e. 在语言模型和 diffusion 上都实验过，收敛效果好，效果接近甚至超过 PPO。

缺点：

a. 只能用 pairwise preference，无法利用 reward model 的连续分数信息。

b. 依赖数据质量：如果 pair 样本不准（reward model 打分有偏），优化方向也会出问题。

c. 对 fine-grained 的 reward shaping 不如 PPO 强。

3.2.2 常见工具

(1) TRL (Transformer Reinforcement Learning)

Hugging Face 推出的库，用来做语言模型的 RLHF。

定位：

- 面向语言模型（LM/LLM）。
- 提供 PPO、DPO、奖励建模等现成实现。
- 核心是基于 transformer 语言模型的强化学习训练。

优点：

- a. 成熟生态，文档全，和 HF 模型无缝衔接。
- b. RLHF 的开箱即用工具。
- c. 在 NLP 社区应用最广（ChatGPT 式训练）。

缺点：

- a. 主要针对语言模型，对 diffusion 这类视觉生成模型支持不足。
- b. 不适合大规模分布式训练，更多是研究/中小规模实验。

(2) VERL (Versatile RL library, by ByteDance)

字节开源的通用分布式强化学习库。

定位：

- 面向大模型训练（语言+视觉）。
- 提供分布式、高效的 RLHF 训练框架。
- 支持 DPO、PPO、GRPO 等多种算法。
- 专门优化了大规模分布式（多机多卡）训练场景。

优点：

- a. 高性能，支持数百张 GPU 并行。
- b. 针对大模型的内存优化、流水线调度，比 TRL 更适合工业级别训练。
- c. 可以直接跑 DiffusionDPO，适配视觉模型。

缺点：

- a. 上手门槛较高，需要熟悉分布式环境。
- b. 文档/社区规模不如 HuggingFace TRL。

(3) DiffusionDPO

是一种算法（Direct Preference Optimization 的扩展版），不是库。

定位：

- 专门为 扩散模型（Diffusion Models） 设计的 RLHF 方法。
- 利用人类偏好对（或奖励模型生成的偏好对），直接训练扩散模型。
- 解决了 PPO 在 diffusion 上训练复杂、显存爆炸的问题。

优点：

- a. 高效、稳定，不需要 reward model 在线参与。
- b. 特别适合 LoRA 微调 + 资源有限的场景。
- c. 已经被验证能提升图像质量和用户偏好对齐度。

缺点：

- a. 只能用 pairwise preference，不能利用 reward 的连续分数。
- b. 效果依赖数据质量（pair 样本是否准确）。

3.3 基于 DiffusionDPO 的 LoRA SFT

主要由于显卡限制，故基于 DiffusionDPO 展开 LoRA SFT。

3.3.1 LoRA 的原理

LoRA (Low-Rank Adaptation) 并不是对整个模型参数进行更新，而是只在特定的 线性层 上插入一个低秩矩阵分解（ $W \approx W_0 + \Delta W$ ，其中 $\Delta W = A \cdot B$ ，秩 $r \ll d$ ）。

这样可以在保留原始大模型参数冻结不动的前提下，通过少量新增参数（几百万级，而不是几十亿）来完成模型的高效微调。

3.3.2 DiffusionDPO

DiffusionDPO 核心逻辑就是：

$$\mathcal{L}_{\mathcal{DP}O} = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \cdot (\log \pi_{\theta}(y_w|x) - \log \pi_{\theta}(y_l|x)) \right) \right]$$

y_w : 获胜样本

y_l : 失败样本

π_θ : 你微调后的扩散模型

β : 平衡因子（你训练时设置过）

3.3.3 实验配置

`--lora_target_modules to_q to_v`

LoRA 仅作用在 SD3Transformer2DModel 中 Self-Attention 模块的 Query (Q) 和 Value (V) 投影层。

在 transformer 的注意力机制中，Q, K, V 是通过 Linear 层从输入 hidden state 映射出来的：

$$Q = XW_Q, K = XW_K, V = XW_V$$

选择的 `to_q` 和 `to_v`，意味着：

Q 投影矩阵 (W_Q): 决定模型如何从输入特征中提取“关注方向”。

V 投影矩阵 (W_V): 决定注意力分数加权后的输出表示。

这两个部分对生成任务的内容与细节表达影响最大，所以它们是最常见的 LoRA 微调目标。Q 和 V 的参数量相对适中，又直接影响注意力分布和特征重构能力。很多开源经验表明，仅 Q+V LoRA 就能达到与全量微调接近的效果。如果加上 `to_k`、`ffn` 等模块，LoRA 参数会暴涨，显存消耗和训练时间大幅增加。

3.4 显存优化

3.4.1 xFormers 加速

做法：在 DiffusionPipeline 或 StableDiffusion3Pipeline 加载完成后，调用 `pipe.enable_xformers_memory_efficient_attention()`。

作用机理：

替换原生的注意力计算（standard attention）为 xFormers 提供的 memory-efficient attention 算法。将标准 $O(N^2)$ 的注意力显存开销优化为近似的分块计算/低精度计算，极大降低了中间激活的显存占用。

效果：

显存节省约 20%~30%，尤其在 batch size 较大时显著。注意力计算加速，整体训练速度提升约 1.1~1.3 倍。

3.4.2 CPU Offload

做法：

调用 `pipe.enable_model_cpu_offload()` 或使用 `Accelerate` 提供的 `dispatch_model`。将部分模块（如 VAE 解码器、Text Encoder 等）在不使用时自动转移到 CPU，仅在前向传播需要时转回 GPU。

作用机理：

避免整网参数常驻 GPU，仅在计算时占用显存。相当于在 计算和显存之间做分时调度。

效果：

在 batch size 较大或梯度累积较多的情况下，显存占用可减少 2~6GB。代价是 CPU-GPU 数据传输开销，导致速度略微下降（训练速度降低约 5%~10%）。

3.4.3 模型量化 (Quantization)

做法：

使用 `BitsAndBytesConfig` 将部分权重量化为 4-bit (NF4) 或 8-bit (int8) 格式。在 LoRA 微调中，通常主干模型参数保持量化，而 LoRA 参数保持全精度 (float16/float32)。

作用机理：

通过低比特宽度存储参数，显著降低模型的 权重存储与显存占用。同时利用 GPU 的 tensor core (int8/FP16 支持) 保持较高的运算吞吐量。

效果：

显存占用下降 30%~50%。模型收敛速度接近全精度，精度损失可忽略。

4 实验结果

我们基于 geneval 提供的 553 个 prompt 进行生图，每个图片生成四张图片，结果如下。

指标类别	SD3.5-Medium Baseline	我们的结果	变化趋势
总图片数	2212	2212	—

总提示数	553	553	—
正确图片率	67.54%	68.17%	↑ +0.63%
正确提示率	86.08%	83.91%	↓ -2.17%
单目标任务	100.00% (320/320)	99.69% (319/320)	↓ -0.31%
双目标任务	84.85% (336/396)	87.88% (348/396)	↑ +3.03%
计数任务	56.56% (181/320)	66.88% (214/320)	↑ +10.32%
颜色识别	82.18% (309/376)	83.24% (313/376)	↑ +1.06%
位置关系	26.00% (104/400)	23.50% (94/400)	↓ -2.50%
颜色属性	61.00% (244/400)	55.00% (220/400)	↓ -6.00%
整体分数	0.68432	0.69364	↑ +0.00932
